Recommender Systems - Course Project Report

Part 1 submission: data, protocol and baselines

Team	Group 26
Members	Siva sandeep Atmakuru
For Part 3 track: select one	A / B / C
GitHub repository link	https://github.com/Siva-Sandeep7/Recommender_System-Group_26.git

1. Part 1 - Data, Protocol and Baselines

1.1 Data characterization

MovieLens 1M contains 1,000,209 ratings from 6,040 users on 3,706 rated movies (density 4.47%, far denser than the 1% or less typical of the settings in Lecture 1.1). Users have at least 20 ratings (median 96, mean 165.6, maximum 2,314); items have a median of 124 ratings (mean 269.9, maximum 3,428, minimum 1). Figure 1 shows the long tail (Lecture 5.1, slide 5): the top 1% of items receive 8.7% of all ratings, the top 10% receive 44.4% and the top 20% receive 65.2% (item-popularity Gini 0.634), while 12.0% of items have fewer than 10 ratings. Ratings are skewed positive (1: 56,174; 2: 107,557; 3: 261,197; 4: 348,971; 5: 226,310; mean 3.58), and 57.5% are 4 or higher.

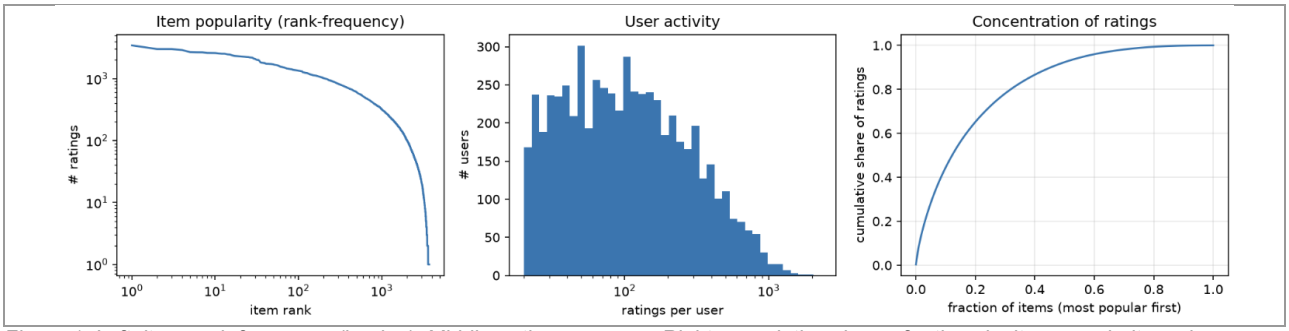


Figure 1. Left: item rank-frequency (log-log). Middle: ratings per user. Right: cumulative share of ratings by item popularity rank.

The most consequential property is the timing. Timestamps record when a rating was entered, from 2000-04-25 to 2003-02-28 (1,039 days), but 904,757 ratings (90.5%) were entered in 2000, the busiest single day (2000-11-20) holds 6.4% of all ratings and the busiest 30 days hold 39.7%. Moreover, 62.9% of users entered all of their ratings within 24 hours and 70.2% within 7 days (median span of a user's history: 0.05 days, about 70 minutes). Most users therefore rated in a single sitting, so a user's "later" ratings are mostly the tail of the same session rather than evidence of preference drift. This directly affects how a temporal split can be built (Section 1.3).

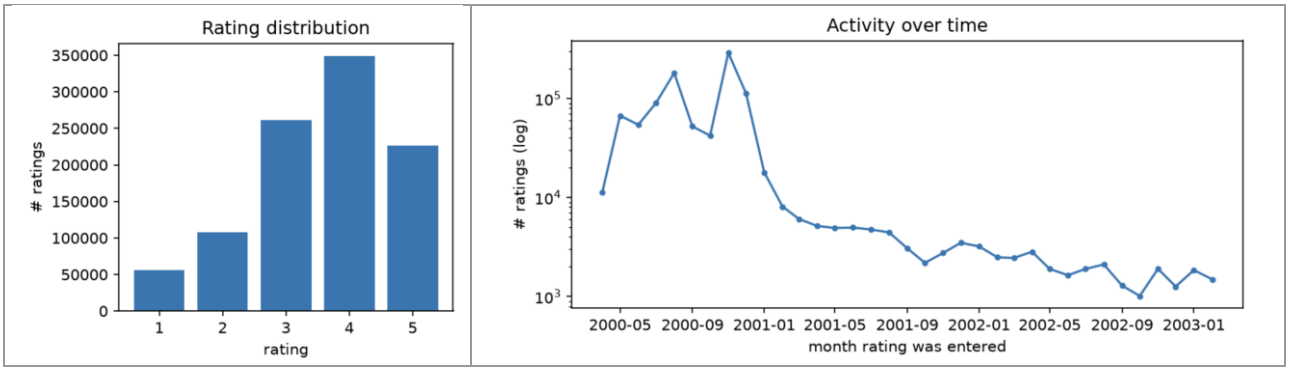


Figure 2. Left: rating distribution. Right: ratings entered per month (log scale).

1.2 Preprocessing and feedback treatment

A rating of 4 or 5 is treated as positive feedback (57.5% of ratings). Positives define the relevant items in evaluation and are the only feedback used to train item-kNN and the positive-only popularity variants. Every rated item, positive or not, counts as seen and is removed from that user's ranking, so a film a user disliked is never recommended again.

The main experiments apply no filtering: MovieLens 1M already guarantees at least 20 ratings per user, and no item threshold was applied, so 0 rows were removed. To test whether filtering matters, we re-ran the final models (settings held fixed) with item thresholds of 5, 10 and 20 ratings, which remove 598, 1,670 and 4,717 rows (0.06%, 0.17%, 0.47%) and 290, 446 and 663 items. ItemKNN NDCG@10 stays between 0.0881 and 0.0883, a spread of 0.0002, against a 0.0299 gap to the best popularity baseline, so filtering does not change any conclusion. Coverage rises from 0.259 to 0.316 as the threshold increases, but the catalogue shrinks at the same time (3,706 to 3,043 items) and we did not separate the two effects, so we do not read this as a model improvement.

1.3 Train/validation/test split

We use a per-user chronological split: for each user, ratings are ordered by timestamp (ties by item id) and the earliest 80% form the training set, the next 10% the validation set and the final 10% the test set (counts rounded down per user). This gives 797,758 training (79.8%), 99,692 validation (10.0%) and 102,759 test (10.3%) ratings. Hyperparameters are tuned with models fitted on the training set and scored on validation. The final models are refit on training plus validation and evaluated once on test.

We first tried a global temporal split (one cut date at the 80th and 90th percentile of timestamps, Lecture 3.1, slide 4). Because most users rate in one burst, the global cut leaves most users without any earlier history: only 492 users could be evaluated on validation, since the rest have no training data and cannot be personalised. The per-user split keeps every user warm and evaluates 5,789 validation and 5,821 test users (all users with at least one positive item in the target period). Its cost is that there is no single cut date, so some test interactions of one user happen earlier than training interactions of another user; the split respects each user's own time order but not a global one, which may make results somewhat optimistic. Together with the burstiness above, test performance mainly measures prediction within a user's session.

1.4 Evaluation protocol

For each evaluated user we rank the full catalogue of 3,706 items, excluding the user's history, with no sampled negatives (as the project specification requires). Ties are broken deterministically by training popularity. Metrics at cutoff 10 (Lecture 3.1, slides 7-8) are averaged over evaluated users: Precision@10 (hits / 10), Recall@10 (hits / number of relevant test items), NDCG@10 with binary gains and the ideal DCG computed over min(number of relevant items, 10), and catalogue coverage@10 (Lecture 3.1, slide 9: distinct items appearing in any evaluated user's top 10 divided by 3,706). We implemented the metrics ourselves and verified NDCG@10 against scikit-learn's ndcg_score (difference below 1e-16) and Precision/Recall against their direct definitions on a random sample. Intervals are 95% percentile bootstrap intervals over users (1,000 resamples, seed 42). Both baselines are deterministic, so the bootstrap is the only source of randomness in Part 1. All tuning uses validation only (Lecture 3.1, slide 10).

1.5 Baseline methods and settings

Item-based kNN (Lecture 1.2, slides 5-10) uses the binary positive user-item matrix. The score of item j for user u is the sum of the similarities between j and the items in u's positive history, where each target item keeps only its k most similar items and negative similarities are clipped to 0. We evaluated cosine and Jaccard similarity on the binary matrix and adjusted cosine (ratings centred by each user's mean), each with k in {10, 20, 50, 100, 200, 500}: 18 configurations. Cosine with k = 200 was best on validation (NDCG@10 0.0915), ahead of cosine k = 500 (0.0910) and Jaccard k = 100 (0.0897); adjusted cosine was worst (best 0.0815). The optimum is inside the grid, but the top three settings are close and we did not compute intervals on validation, so we do not claim cosine k = 200 is strictly better than its neighbours.

Most Popular has no parameters to fit but several choices. We report four variants: all-time or recent-window counts (windows of 7, 30, 90, 180 and 365 days ending at the latest timestamp in the fitted data, window chosen on validation) crossed with counting all ratings or only positive ones (12 configurations in total; selected windows 180 days for all ratings and 365 days for positive-only).

1.6 Results and analysis

Table 1 reports test results for the validation-selected setting of each model (the full grid is in tuning_log.csv of the repository and in the Results Sheet).

Table 1. Test results on 5,821 users; mean with 95% bootstrap interval over users in brackets (coverage has no interval).

Model (selected setting)	P@10	R@10	NDCG@10	Coverage@10
MostPop, all-time, all ratings	0.0445 [0.0422, 0.0467]	0.0479 [0.0452, 0.0507]	0.0569 [0.0539, 0.0601]	0.0402
MostPop, all-time, positive only	0.0455 [0.0431, 0.0479]	0.0493 [0.0465, 0.0525]	0.0584 [0.0553, 0.0616]	0.0391
MostPop, recent 180 d, all ratings	0.0370 [0.0353, 0.0388]	0.0432 [0.0407, 0.0458]	0.0465 [0.0441, 0.0489]	0.0305
MostPop, recent 365 d, positive only	0.0380 [0.0360, 0.0400]	0.0387 [0.0364, 0.0411]	0.0465 [0.0439, 0.0490]	0.0291
ItemKNN (cosine, k = 200)	**0.0631** [0.0604, 0.0657]	**0.0850** [0.0805, 0.0892]	**0.0883** [0.0842, 0.0921]	**0.2593**

- **ItemKNN is the clear winner on accuracy.** Its NDCG@10 of 0.0883 [0.0842, 0.0921] is about 1.5 times that of the best popularity variant, 0.0584 [0.0553, 0.0616], and the intervals do not overlap; Precision@10 (0.0631 vs 0.0455) and Recall@10 (0.0850 vs 0.0493) agree. This comparison uses two unpaired intervals; Part 2 will use paired tests across users.
- **Popularity is almost a single list.** Coverage@10 is 0.259 for ItemKNN but only 0.029-0.040 for the popularity variants, meaning they put roughly 110-150 distinct movies in front of all users.

- **All-time popularity beats recent-window popularity** (NDCG@10 0.0584 vs 0.0465, non-overlapping intervals), whereas counting positives only vs all ratings makes no detectable difference all-time (0.0584 vs 0.0569, overlapping intervals). A possible reason, which we have not tested, is that with bursty timestamps a recent window is a small and noisy sample of what users watch.
- **No sign of overfitting to validation:** ItemKNN scored 0.0915 on validation and 0.0883 on test, and the best popularity variant 0.0585 and 0.0584.

Limitations: the per-user split can be optimistic (Section 1.3), test performance is dominated by within-session behaviour, and Part 1 reports a single data split rather than repeated splits.

5. Contribution Statement

SIVA SANDEEP ATMAKURU was the only team member and carried out all Part 1 work. This included setting up the environment, running the pipeline on MovieLens 1M, reading the results, and deciding to switch to a per-user chronological split after the global split left only 492 validation users evaluable. [Your name] also checked the numbers against the outputs, assembled the report and filled in the results workbook. The code and the first draft of the report text were produced with the help of Claude; SIVA SANDEEP ATMAKURU ran and reviewed them.

6. Use of AI Assistants

Claude (Anthropic) was used to write the Part 1 code (data loading, split, baselines, metrics, plots and the script that generates the report numbers), to debug it, and to draft this report from the team's results. The team ran the code and reviewed the results and text.

References

F. M. Harper and J. A. Konstan. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems, 5(4), 2015.

B. Sarwar, G. Karypis, J. Konstan and J. Riedl. Item-based collaborative filtering recommendation algorithms. Proceedings of WWW, 2001.

Course lecture slides, Recommender Systems: Lectures 1.1, 1.2, 3.1, 5.1.